



南开大学
Nankai University

南开大学

计算机学院和密码与网络空间安全学院

《软件工程》课程作业

软件设计

姓名：陆皓喆

学号：2211044

专业：信息安全

指导教师：刘健

2025 年 4 月 21 日

目录

1 引言	3
1.1 编写目的	3
1.2 项目背景	3
1.3 使用工具	3
2 用例图	4
2.1 用例图简介	4
2.2 用例图构建	5
2.2.1 用户管理子系统	5
2.2.2 数据处理与分析子系统	6
2.2.3 数据可视化子系统	7
2.2.4 报警与通知子系统	8
2.3 用例图解释	8
2.3.1 用户管理子系统	8
2.3.2 数据处理与分析子系统	8
2.3.3 数据可视化子系统	9
2.3.4 报警与通知子系统	9
3 活动图	9
3.1 活动图简介	9
3.2 活动图构建	10
3.2.1 用户注册和登录活动图	11
3.2.2 数据收集和分析活动图	12
3.2.3 可视化模块活动图	13
3.2.4 报警与通知活动图	14
3.3 活动图解释	15
3.3.1 用户登录和注册活动图	15
3.3.2 数据收集和分析活动图	15
3.3.3 可视化模块活动图	15
3.3.4 报警与通知活动图	15
4 类图	16
4.1 类图简介	16
4.2 类图构建	17
4.3 类图解释	17
4.3.1 类图中实例解释	17
4.3.2 类图中类间关系解释	21
5 顺序图	21
5.1 顺序图简介	21
5.2 顺序图构建	22
5.2.1 用户登录	22

5.2.2	数据采集	23
5.2.3	报警设置	24
5.2.4	系统管理	25
5.3	顺序图解释	26
5.3.1	用户登录	26
5.3.2	数据采集	26
5.3.3	报警设置	26
5.3.4	系统管理	26
6	协作图	27
6.1	协作图简介	27
6.2	协作图构建	27
6.2.1	用户登录	27
6.2.2	数据采集	28
6.2.3	报警设置	28
6.2.4	系统管理	29
6.3	协作图解释	29
6.3.1	用户登录	29
6.3.2	数据采集	29
6.3.3	报警设置	30
6.3.4	系统管理	30
7	状态图	30
7.1	状态图简介	30
7.2	状态图构建	31
7.3	状态图解释	32
8	构件图	32
8.1	构件图简介	32
8.2	构件图构建	33
8.3	构件图解释	33
9	部署图	35
9.1	部署图简介	35
9.2	部署图构建	36
9.3	部署图解释	36

1 引言

软件设计文档的意义

设计文档是软件工程设计中的重要组成部分,是对一个技术问题的解决方案的系统性描述。设计文档的目的,是阐明设计的总体思想和设计中考虑的权衡点。

作为一名软件工程师,我们的工作本质不仅仅是编写程序代码,而是解决真正的问题。因此,相比最终的程序代码,文字形式的设计文档,在早期能够更加简明扼要地传达信息,便于让读者理解问题,找到解决方案。

1.1 编写目的

本文档旨在为“智慧海洋牧场可视化系统”提供一份全面且清晰的软件设计方案,确保系统的架构、功能模块以及技术实现符合预定的需求。通过详细的设计文档,项目开发团队能够准确把握系统的整体结构和各个模块的实现路径,确保开发过程中的每个步骤都能高效且符合需求标准。

本文将通过 UML (统一建模语言) 图示方法,使用例图、活动图、类图、顺序图、协作图、状态图、构件图和部署图,系统地展示软件设计的各个方面。这些图示不仅帮助开发人员理解系统各模块的功能和接口,还能帮助团队成员从整体上把握系统设计思路,从而有效预防设计偏差和实现过程中可能出现的技术问题。

本文档的主要目标是为开发人员提供清晰的技术方案和具体的编码指导。文档将详细讨论各模块的设计原理、数据流向、交互逻辑及性能优化等方面,确保在开发过程中所有技术决策和实现方法都有充分的理论依据和实用性。

UML 统一建模语言

UML (统一建模语言) 是一种标准化建模语言,通过图形符号辅助开发人员设计、展示和文档化软件系统。它广泛应用于大型复杂系统建模,是面向对象软件开发的关键工具,可促进团队沟通、设计探索及架构验证。

1.2 项目背景

中国在全球水产养殖领域占据重要地位,但现有养殖体系仍面临诸多挑战。相比之下,国际先进养殖业已广泛应用智能水质传感器和在线监测系统,实现对养殖环境的全方位管控,并借助自动化设备优化生产流程,在提升效率的同时降低环境影响。为改善现状,中国在“十四五”规划中提出建设“物联网 + 海洋牧场”的发展战略,将运用物联网、大数据分析 & 云计算等数字化技术,打造智能化、实时化的养殖监测平台,通过精准调控养殖参数来提升生产效能并降低疫病风险。

1.3 使用工具

本文档使用 Overleaf 完成编辑,使用的语言是 latex,文档中所有图示均使用 GitMind 进行绘制。

关于 GitMind

GitMind 具备思维导图、鱼骨图等多种图形绘制功能,含 AI 智能生成、大纲视图、演示模式等特色功能,能满足不同用户的绘图与协作需求。

2.2 用例图构建

针对“智慧海洋牧场可视化系统”，我们将其分为四个子系统：用户管理子系统、数据处理与分析子系统、数据可视化子系统和报警与通知子系统。下面是我们小组为这四个子系统构建的用例图，我将在后面的部分中首先展示我做的图，然后再对图的部分细节做一下说明。

2.2.1 用户管理子系统

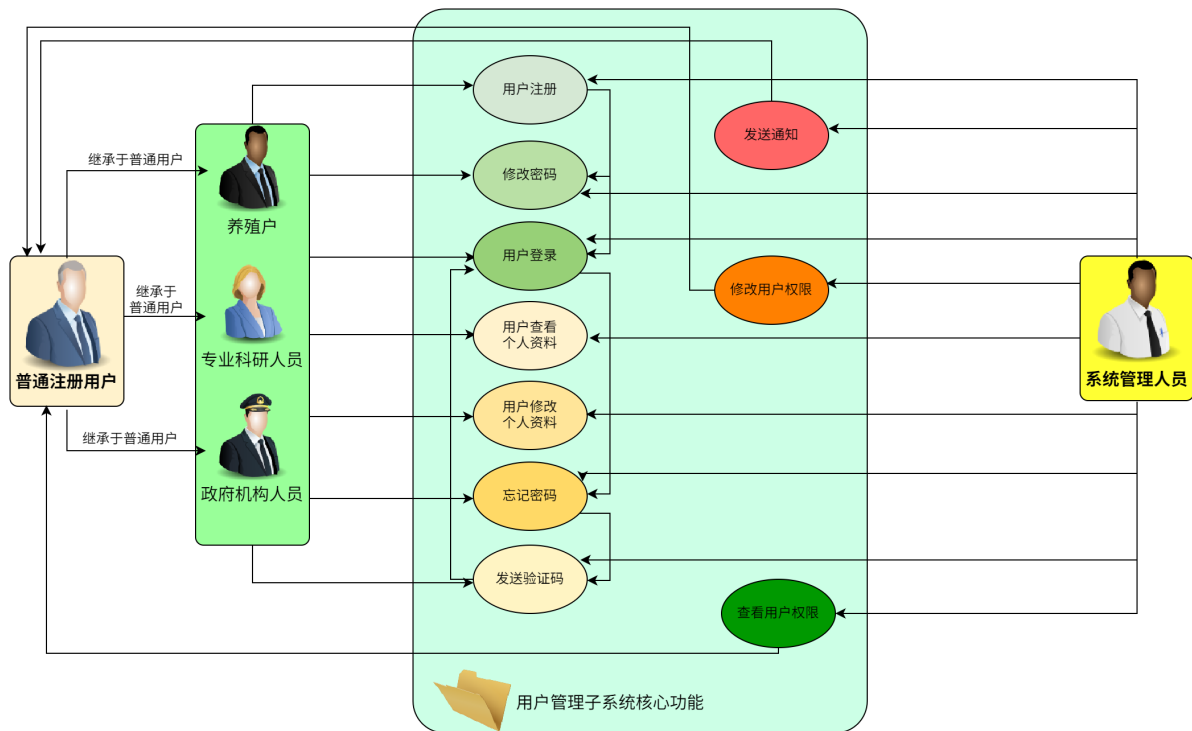


图 2.3: 用户管理子系统

2.2.2 数据处理与分析子系统

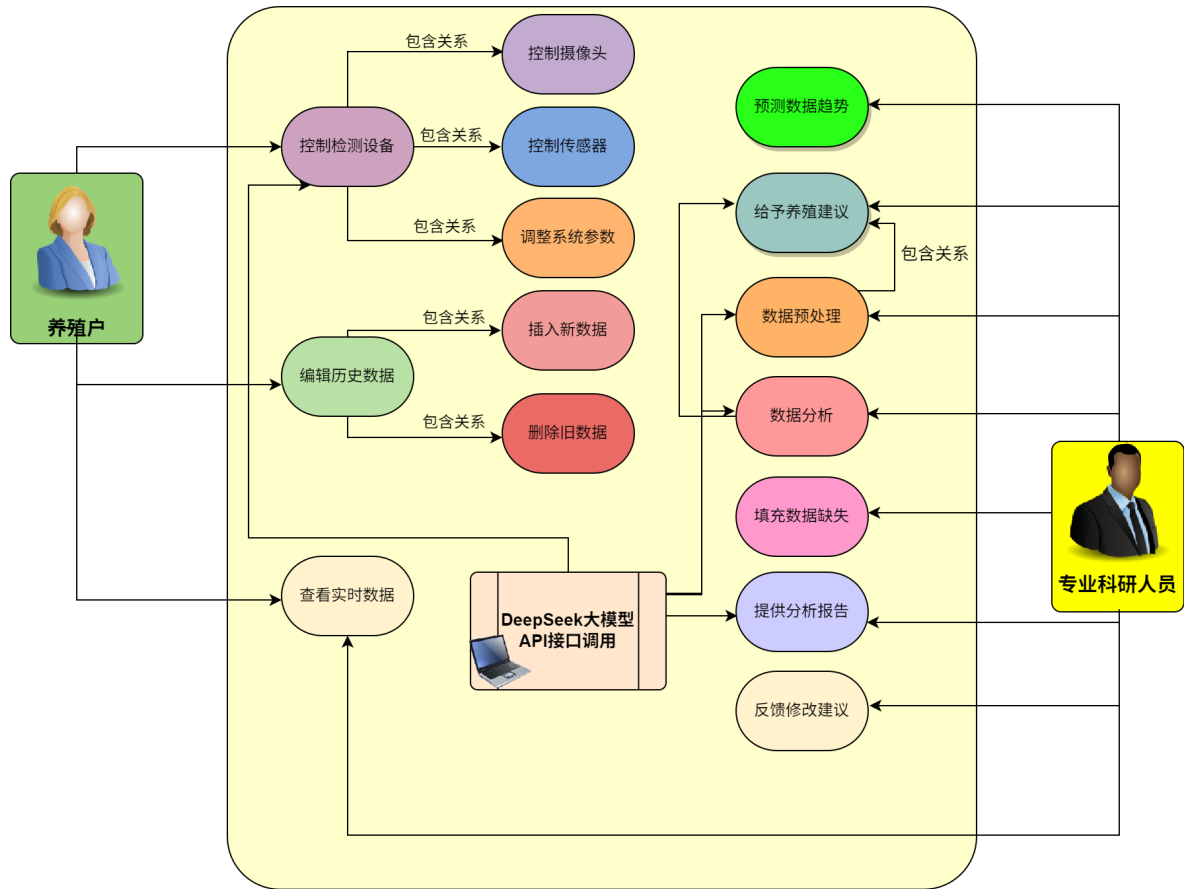


图 2.4: 数据处理与分析子系统

2.2.3 数据可视化子系统

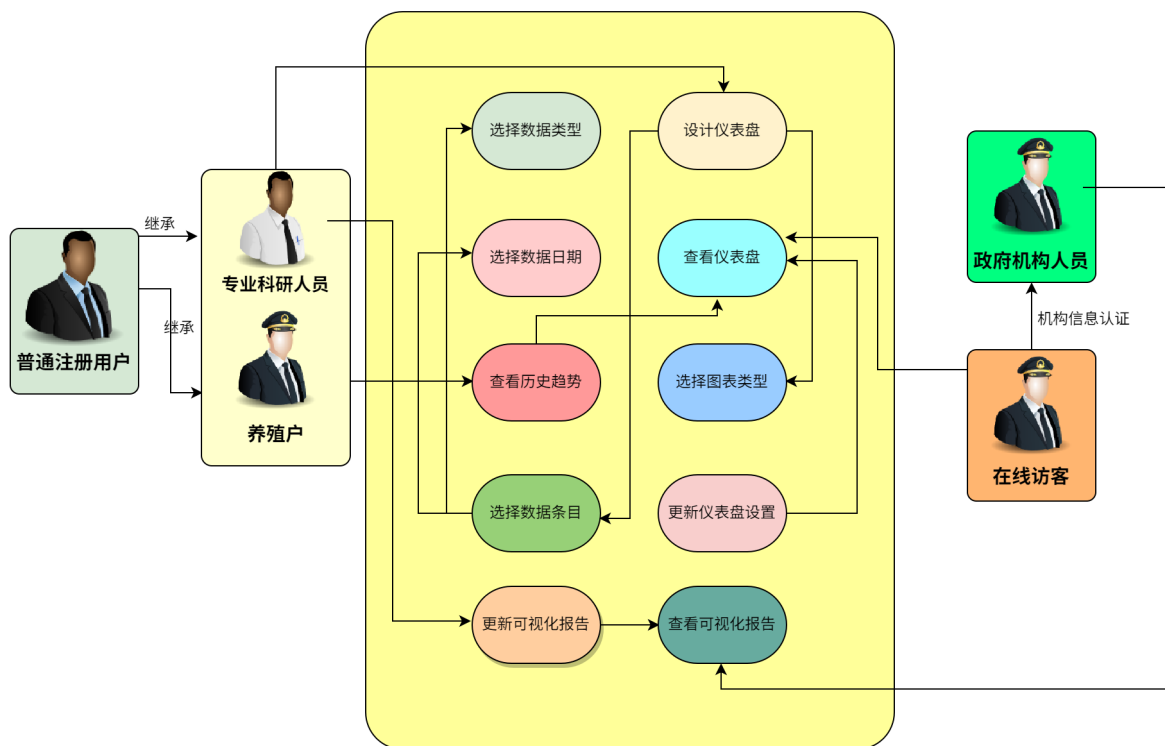


图 2.5: 数据可视化子系统

2.2.4 报警与通知子系统

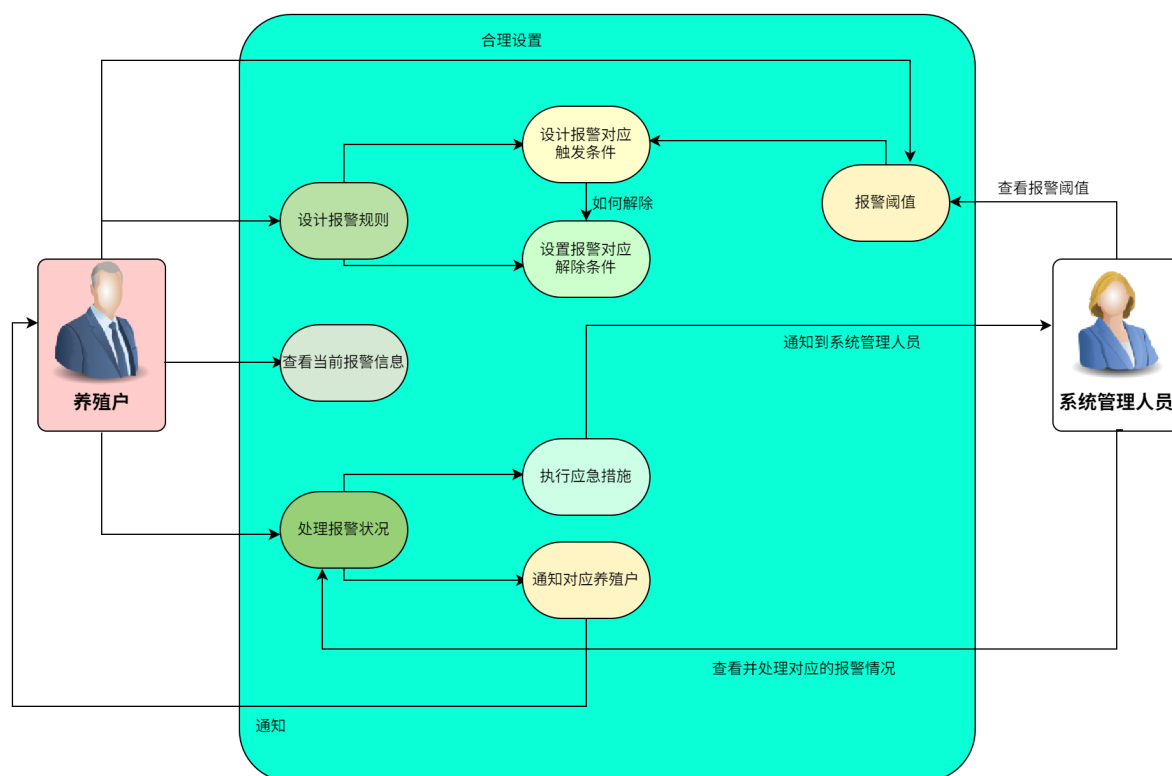


图 2.6: 报警与通知子系统

2.3 用例图解释

2.3.1 用户管理子系统

首先，我们开设一个普通注册用户的大类，然后养殖户、专业科研人员和政府机构人员都可以继承于我们的普通注册用户。对应的，我们只需要进行对应的身份认证就可以完成身份的切换了。这三类用户具有用户注册、修改密码、用户登录、查看个人资料、修改个人资料、忘记密码以及发送验证码等功能。

对应的，我们的系统管理人员除了拥有上述功能之外，还拥有发送通知、修改用户权限、查看用户权限等功能。

2.3.2 数据处理与分析子系统

这张数据处理与分析子系统的用例图呈现了养殖户和专业科研人员与系统的交互。养殖户作为养殖一线的实际操作者，能够对控制检测设备进行操作，其中涵盖控制摄像头来监控养殖环境、控制传感器以精准采集数据，还能调整系统参数适配养殖需求；在数据管理上，可编辑历史数据，进行插入新数据和删除旧数据的操作，也能随时查看实时数据掌握动态。专业科研人员凭借专业知识，通过预测数据趋势洞察走向，给予养殖建议助力科学养殖，开展数据预处理、数据分析、填充数据缺失等专业工作，还会提供分析报告并反馈修改建议，而系统借助 DeepSeek 大模型 API 接口调用实现各项功能，保障整个子系统的的历史数据处理与分析流程顺畅运作。

2.3.3 数据可视化子系统

这张数据可视化子系统的用例图，其设计思路围绕不同类型用户及其需求展开。以普通注册用户为源头，基于其进一步细分出专业科研人员、养殖户，他们因具备不同专业背景和业务需求而继承普通注册用户身份，专业科研人员需运用专业知识进行数据深度处理，故而被赋予设计仪表盘等专业操作权限；养殖户则侧重于养殖业务相关数据，如查看历史趋势。在线访客作为临时用户，可进行基础的可视化操作；政府机构人员经信息认证后，能获取规范数据展示用于监管。

在用例方面，围绕数据处理与展示构建。“选择数据类型”“选择数据日期”“选择数据条目”让用户能精准筛选所需数据；“设计仪表盘”“查看仪表盘”“更新仪表盘设置”为用户提供定制和查看数据展示界面的功能；“选择图表类型”助力用户以合适图表呈现数据；“查看历史趋势”满足对数据变化规律的洞察需求；“更新可视化报告”“查看可视化报告”为用户提供规范的数据成果以供决策参考。

2.3.4 报警与通知子系统

在报警与通知子系统的设计中，从参与者角度，确定养殖户和系统管理人员为核心对象。养殖户作为养殖一线人员，需具备设计报警规则的能力，涵盖触发和解除条件设定，以便依据实际养殖场景灵活定义报警标准；同时赋予其查看当前报警信息、处理报警状况的权限，处理时可执行应急措施并通知相关人员，保障能快速响应异常。系统管理人员负责整体把控，可查看报警阈值，并在接收报警通知后进行处理，确保系统稳定运行。整体设计围绕养殖异常的监测、预警及处理流程展开，保证信息及时传递与问题有效解决。

3 活动图

3.1 活动图简介

活动图 (activity diagram) 是一种用于表示工作流程的图形化建模工具。这种图表主要由各种活动和动作元素组成，并且支持分支选择、循环迭代以及并行处理等复杂流程结构。在统一建模语言 (UML) 规范中，活动图主要应用于对计算过程和组织流程（即工作流）进行建模，同时也能够描述相关活动之间的数据流动关系。虽然活动图的主要功能是展示控制流程，但它同样可以表示数据在不同存储位置之间的传输过程。

活动图的主要元素包括：

- **活动节点**：表示工作流中的执行步骤或任务。
- **动作节点**：流程中的基本操作单元，不可再分解。
- **开始节点**：标识流程的起始点（实心圆）。
- **结束节点**：标识流程的终止（实心圆套圆圈或实心圆）。
- **决策节点**：菱形符号，表示条件分支。
- **分叉节点**：粗水平线，表示并行流程的开始。
- **控制流**：实线箭头，表示活动的执行顺序。
- **泳道**：垂直/水平分区，划分不同责任主体。

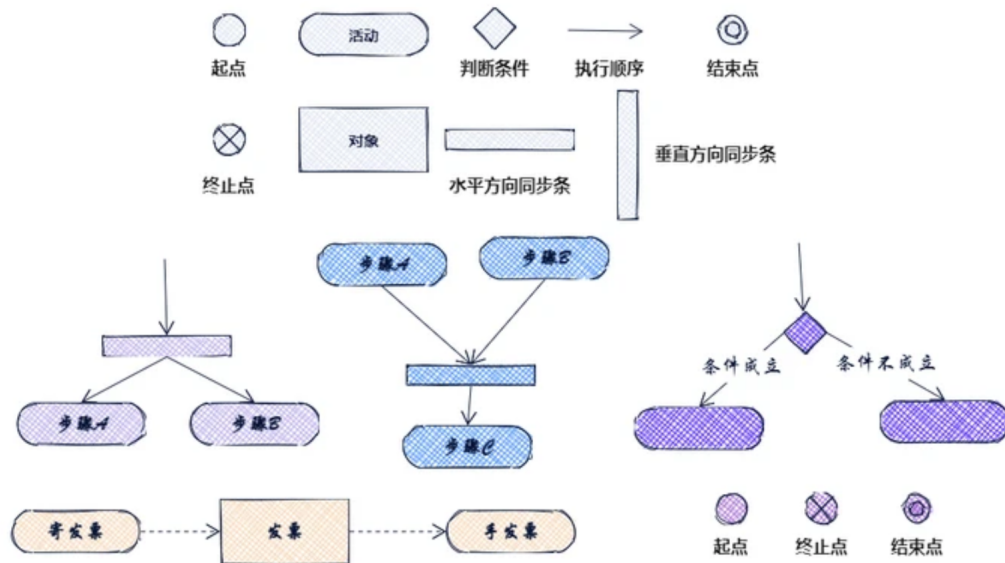


图 3.7: UML 活动图包含的元素图示

3.2 活动图构建

针对“智慧海洋牧场可视化系统”，我们将系统活动分为四个活动图：用户注册和登录活动图、数据收集和分析活动图、可视化模块活动图和报警与通知活动图。下面是我们小组绘制的 4 张活动图：

3.2.1 用户注册和登录活动图

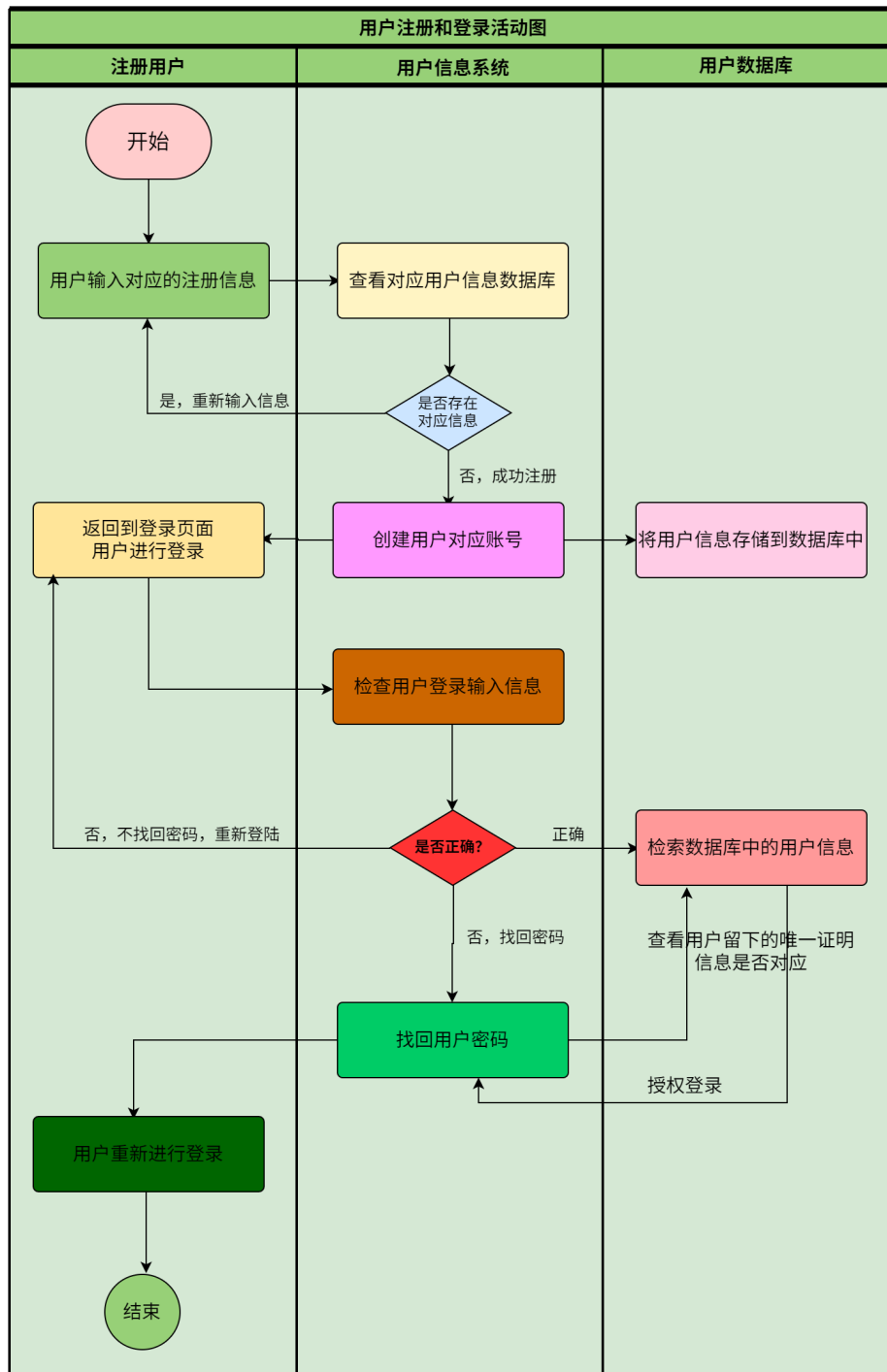


图 3.8: 用户注册和登录活动图

3.2.2 数据收集和分析活动图

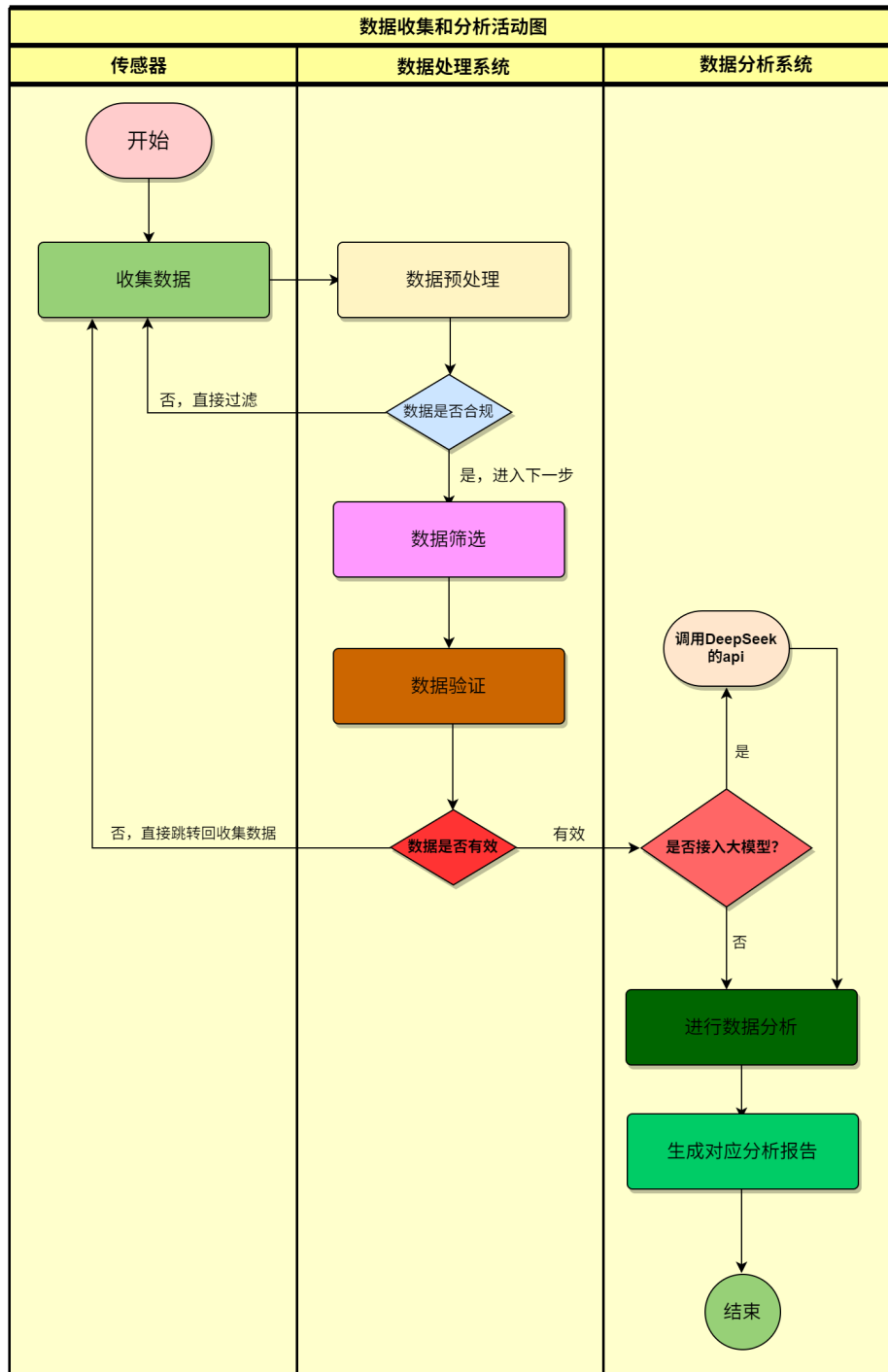


图 3.9: 数据收集和分析活动图

3.2.3 可视化模块活动图

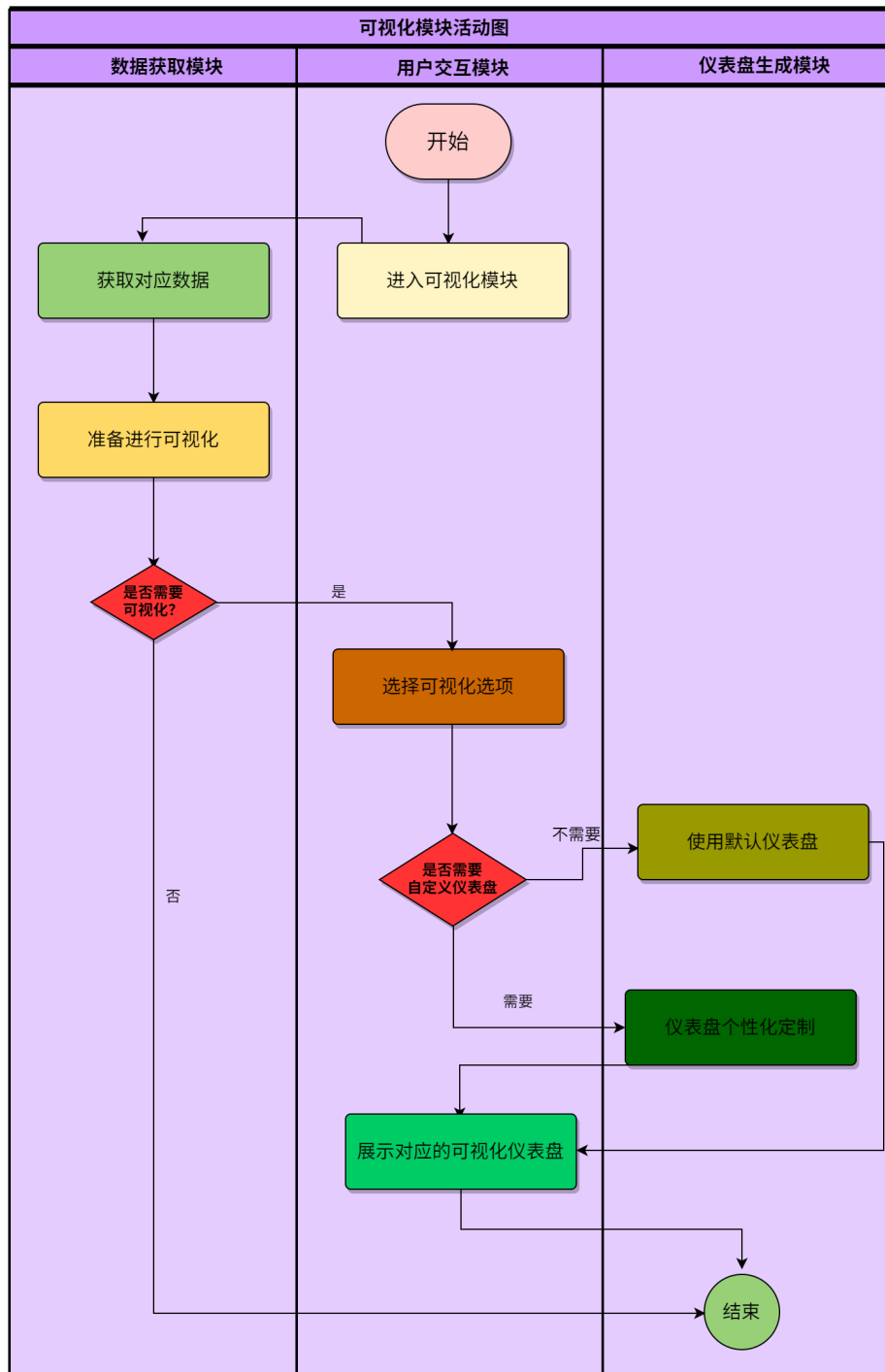


图 3.10: 可视化模块活动图

3.2.4 报警与通知活动图

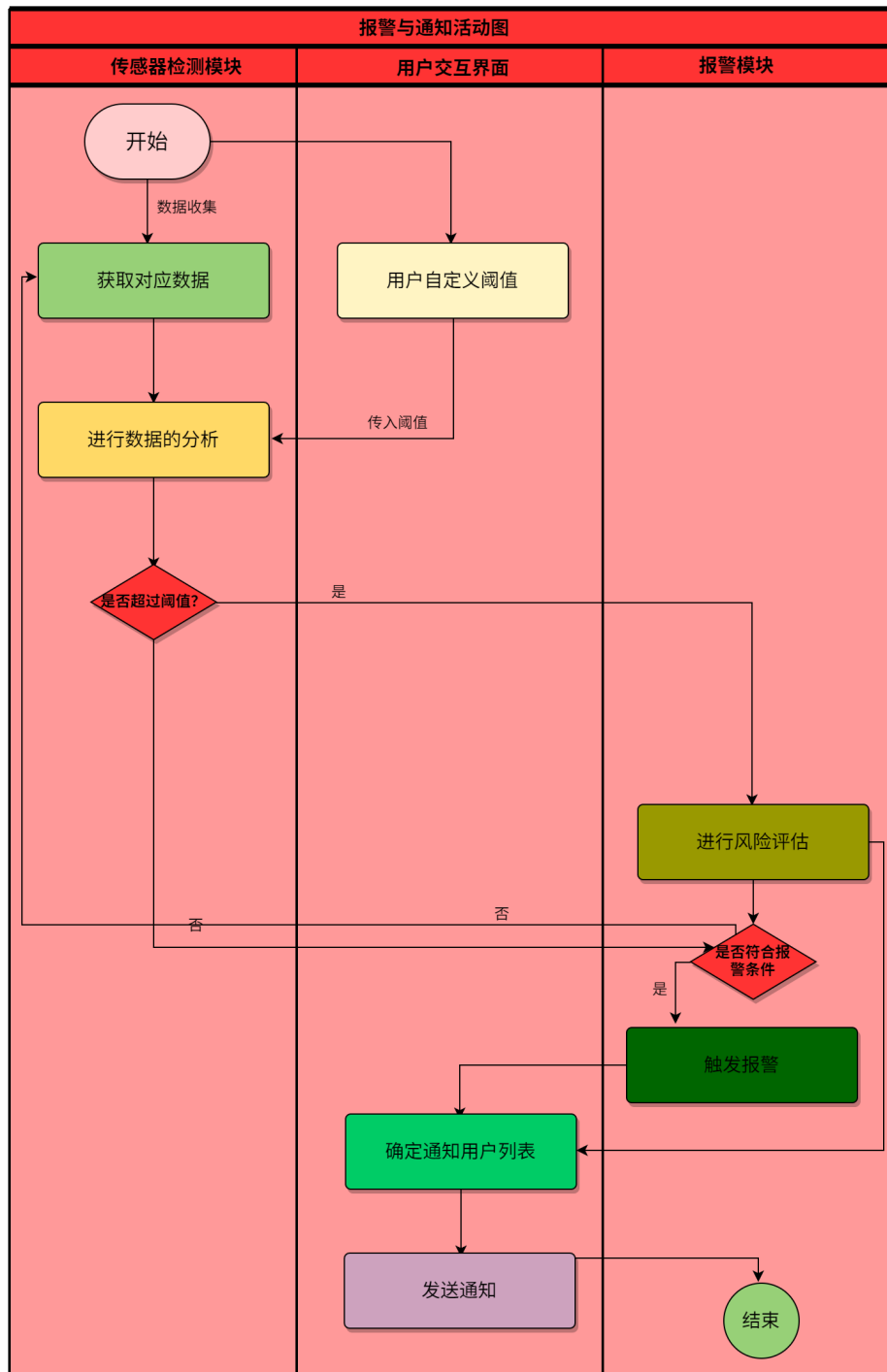


图 3.11: 报警与通知活动图

3.3 活动图解释

3.3.1 用户登录和注册活动图

用户登录和注册活动图如图3.8所示，活动图主要涵盖“注册用户”“用户信息系统”和“用户数据库”三个泳道。

活动起始，用户在注册界面输入注册信息，接着用户信息系统对这些信息的有效性展开验证。若注册信息不满足规范（比如用户名不符合要求），则返回至输入注册信息环节；若信息有效，用户信息系统便创建用户账号。

账号创建后，控制流异步分叉，分别流向输入登录信息和存储用户信息两个方向。如此设计，旨在异步处理用户交互与数据库访问，避免因用户长时间无操作致使系统阻塞。随后，分支流合并。经过判断环节，验证登录信息中的用户名和密码是否匹配。若密码错误，用户可返回输入登录信息界面，或者选择找回密码，通过用户数据库检索用户存储信息。待输入正确密码后，进入登录成功界面，活动至此结束。

3.3.2 数据收集和分析活动图

数据收集和分析活动图如图 3.9 所示，活动图主要由“传感器”“数据处理系统”和“数据分析系统”三个泳道组成。

首先，传感器收集检测区域内气象、温度、含氧量、鱼群尾数等信息，将原始数据传输至数据处理系统。数据处理系统先进行数据预处理，随后判断数据是否合规，不合规数据直接过滤，合规数据进入数据筛选环节。筛选后的数据经数据验证，若数据无效则转回传感器重新收集，有效数据发送至数据分析系统。

在数据分析系统中，先判断是否接入 DeepSeek 大模型。若接入，调用其 API 对数据进行分析；若未接入，则直接进行数据分析。无论是否接入大模型，最终都对数据进行分析并生成总体分析报告，活动结束。

3.3.3 可视化模块活动图

可视化模块活动图如图 3.10 所示，该活动图主要由“数据获取模块”“用户交互模块”和“仪表盘生成模块”三个泳道组成。

起始于用户在用户交互模块发送数据可视化请求，数据获取模块接收并处理该请求，获取对应数据并准备可视化。接着判断是否需要可视化，若不需要则流程结束；若需要，用户在用户交互模块选择可视化选项，随后判断是否需要自定义仪表盘。若不需要，仪表盘生成模块调用默认仪表盘；若需要，则进行仪表盘个性化定制。最终在用户交互模块展示可视化仪表盘，流程结束。

可视化模块提供用户自定义选项，增强了系统交互性，提升了用户体验。

3.3.4 报警与通知活动图

报警与通知活动图如图 3.11所示，该活动图由“传感器检测模块”“用户交互界面”和“报警模块”三个泳道组成。

活动开始后，控制流异步分为两支：一支是传感器检测模块进行数据采集；另一支是用户在用户交互界面自定义报警阈值，并将阈值传入传感器检测模块。接着，传感器检测模块对获取的数据进行分析，随后判断数据是否超过阈值。

若数据未超过阈值，返回数据收集步骤；若超过阈值，则进入报警模块进行风险评估。在报警模块中，评估符合报警条件则触发报警，不符合则返回数据收集步骤。触发报警后，在用户交互界面确定通知用户列表，向相关注册用户发送通知，活动结束。

4 类图

4.1 类图简介

UML 类图是面向对象系统的静态结构图，描述类、接口及其关系。包含类名、属性、方法三部分，通过关联、继承、依赖等关系展示系统架构。常用于软件设计阶段，直观表达类间协作，是开发者的核心建模工具。

类图的核心元素包括以下几个：

- **类**：是对一组具有相同属性、操作、关系和语义的对象的抽象，它是面向对象系统组织结构的核心，包括名称部分（Name）、属性部分（Attribute）和操作部分（Operation）。
- **接口**：定义了相同操作和语义的抽象行为，是面向对象的核心规范，包含名称和方法声明（无属性）。类或接口通过实现/继承接口，实现多态和松耦合。
- **关系**：
 - **关联**：描述类之间的持久性连接，表示对象间的长期静态关系，如学生与课程。
 - **依赖**：一个类的变化可能影响另一个类，是临时性的使用关系，如订单依赖支付接口。
 - **泛化**：类之间的继承关系，子类继承父类的属性和方法，如猫与动物的关系。
 - **聚合**：表示整体与部分的弱拥有关系，部分可独立存在，如汽车与轮胎。
 - **组成**：强聚合关系，部分不能脱离整体存在，如公司与部门的关系。
 - **实现**：类实现接口的契约，定义行为规范，如排序类实现 Comparable 接口。

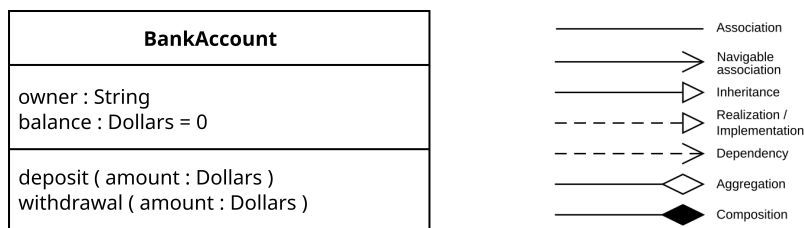


图 4.12: 类图的三个组成部分（左）以及类之间的几种关系（右）

4.2 类图构建

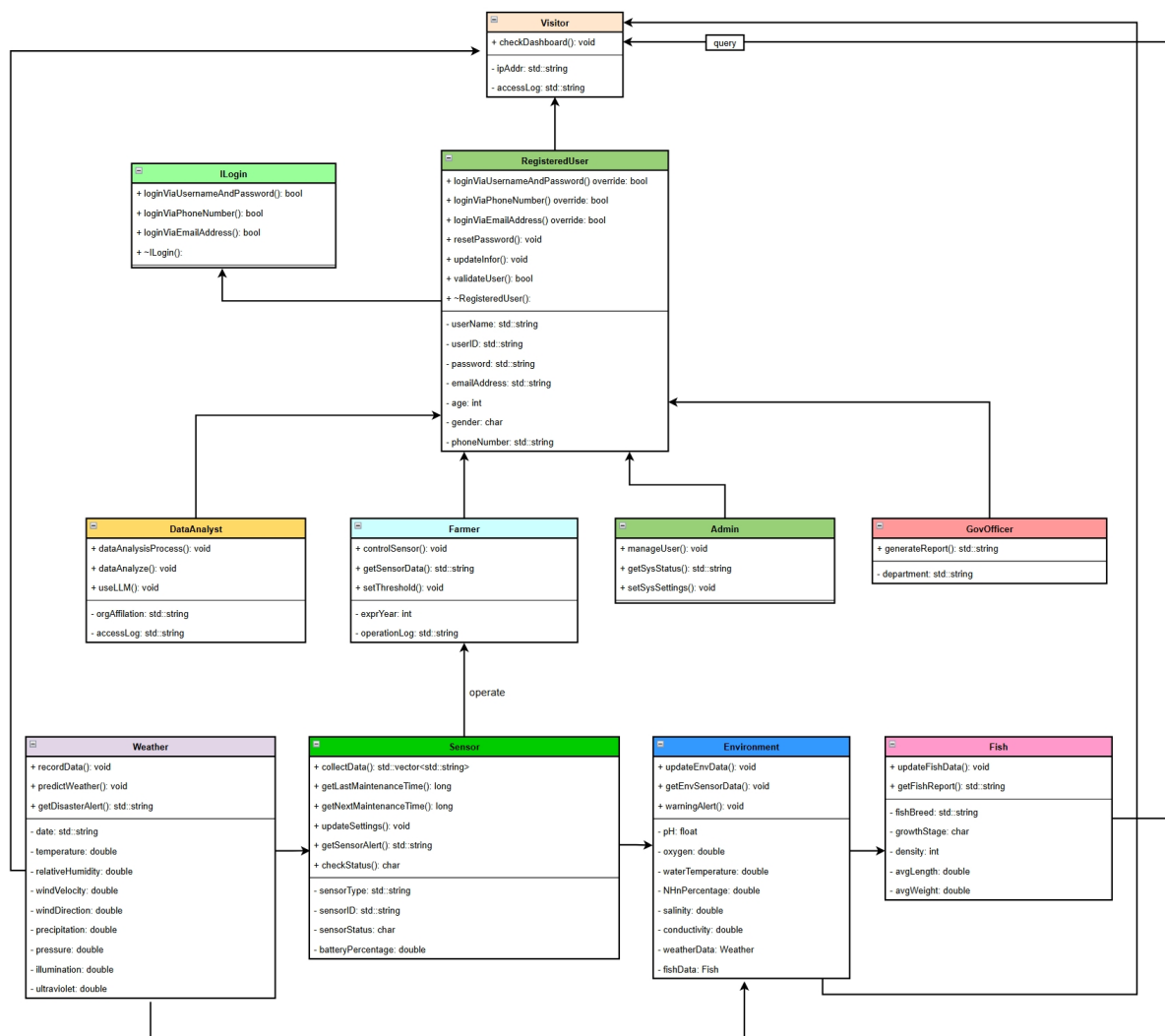


图 4.13: 海洋牧场可视化监测系统类图展示

4.3 类图解释

4.3.1 类图中实例解释

1. **ILogin 接口**：基础的登录功能接口，为上层的注册用户基类提供了模板，待实现。

(a) 方法:

- boolean loginViaUsernameAndPassword(string userName, string password): 使用用户名和密码登录系统, 返回登录成功与否。
- boolean loginViaPhoneNumber(string phoneNumber, string validationCode): 使用手机号和验证码登录系统, 返回登录成功与否。
- boolean loginViaEmailAddr(string emailAddr, string password): 使用邮箱地址和密码登录系统, 返回登录成功与否。

2. **Visitor 类**：访客型用户类，作为其它所有类型用户的父类，具有基础的数据检索的权限。

(a) 属性:

- string ipAddr: 访客的 IP 地址。
- string accessLog: 访客的访问记录日志。

(b) 方法:

- void checkDashboard(): 查看系统仪表盘的基础数据展示。

3. **RegisteredUser** 类: 注册类型用户, 是除了访客外所有用户类型的父类, 它实现了 ILogin 相对应的方法, 具有一些用户普遍具有的信息以及方法。和访客不同的时, 注册类型的用户需要记录一些列和用户相关的属性。

(a) 属性:

- string userName: 用户的登录用户名。
- string userID: 用户的唯一标识 ID。
- string password: 用户的登录密码。
- string emailAddress: 用户的电子邮箱地址。
- int age: 用户的年龄。
- char gender: 用户的性别。
- string phoneNumber: 用户的电话号码。

(b) 方法:

- void resetPassword(string userID, string oldPassword, string newPassword): 重置用户密码, 需验证旧密码。
- void updateInfo(string userID, string newInfo): 更新用户的基本信息。
- boolean validUser(string userID): 验证用户 ID 是否有效。

4. **DataAnalyst** 类: 数据分析师类, 继承自 RegisteredUser 类。该角色负责系统中数据的查看、处理以及分析, 因此他具有父类拥有的数据检索权限。

(a) 属性:

- string orgAffiliation: 所属组织或公司名称。
- string accessLog: 数据访问记录日志。

(b) 方法:

- void dataPreProcess(long timeBegin, long timeEnd, string dataType, string method): 对指定时间段和类型的数据进行预处理。
- void dataAnalyze(long timeBegin, long timeEnd, string dataType, string method): 对指定时间段和类型的数据进行分析。
- void useLLM(long timeBegin, long timeEnd, string dataType): 调用 AI 模型对数据进行预测分析。

5. **Farmer** 类: 养殖户类, 继承自 RegisteredUser 类。该角色负责系统中传感器等设备的监控。他同样具有父类所拥有的数据检索权限, 与其它用户不同的是他还具有传感器数据的检索权限。

(a) 属性:

- int exprYear: 养殖经验年限。
- string operationLog: 设备操作记录日志。

(b) 方法:

- void controlSensor(string sensorID, string instruction): 控制指定传感器的操作指令。
- string getSensorData(string sensorID): 获取指定传感器的监测数据。
- void setThreshold(string sensorID, double threshold): 设置指定传感器的报警阈值。

6. **Admin 类**: 系统管理员类, 继承自 RegisteredUser 类。该角色是系统的管理者, 具有最高的权限, 可以管理其它所有的用户, 同时也可以查看系统实时的运行状况并修改系统设置参数。

(a) 方法:

- void manageUser(string userID, string operation): 管理用户账户, 如创建、删除或修改。
- getSysStatus(): 获取系统当前的运行状态信息。
- setSysSettings(string settingsID, string newSettingsParam): 修改系统的配置参数。

7. **GovOfficer 类**: 政府职员类, 继承自 RegisteredUser 类。该角色属于外部注册人员, 只需要为其提供基本的数据查看和报告生成功能即可。

(a) 属性:

- string department: 所属政府部门名称。

(b) 方法:

- string generateReport(): 生成系统监测数据的汇总报告。

8. **Weather 类**: 传感器监测到的气象类。由于传感器数据可能来自不同日期, 因此在系统中具体日期和气象类相对于一个键值对。

(a) 属性:

- string data: 气象数据的日期。
- double temperature: 当前温度值。
- double relativeHumidity: 相对湿度值。
- double windVelocity: 风速值。
- double precipitation: 降水量值。
- double pressure: 气压值。
- double illumination: 光照强度值。
- double uvIndex: 紫外线指数值。

(b) 方法:

- void recordData(long dateTime): 记录指定时间的气象数据。
- string predictWeather(long dateTime, long period): 预测未来一段时间的气象情况。
- void disasterAlert(): 触发气象灾害警报。

9. **Sensor 类**: 系统的传感器类。通过设备编号唯一确定一个设备, 该类包含传感器的基本信息如经纬度位置、深度和维护时间等。方便专业人员对设备进行监控和及时的维修, 避免意外情况发生。

(a) 属性:

- string sensorType: 传感器类型。
- string sensorID: 传感器唯一标识。
- char sensorStatus: 传感器当前状态。
- float batteryPercentage: 电池剩余电量百分比。
- (double, double) location: 传感器的经纬度位置。
- double depth: 传感器的水下深度。
- long lastMaintenanceTime: 上次维护时间戳。
- long nextMaintenanceTime: 下次维护时间戳。

(b) 方法:

- void updateSettings(string sensorID, string newSettings): 更新传感器的配置参数。
- string getSensorData(string sensorID): 获取传感器的监测数据。
- char checkStatus(string sensorID): 检查传感器的运行状态。

10. **Environment** 类: 传感器监测到的环境数据类。由于传感器数据可能来自不同日期, 因此在系统中具体日期和环境类相对于一个键值对。Environment 类同时又是气象和鱼群的聚合类, 包含另外两个类的对象。

(a) 属性:

- float pH: 水体的 pH 值。
- double oxygen: 溶解氧含量。
- double waterTemperature: 水温值。
- double Ng/N percentage: 氮磷比。
- double salinity: 盐度值。
- double conductivity: 电导率值。
- Weather weatherData: 气象数据对象。
- Fish fishData: 鱼群数据对象。

(b) 方法:

- void updateEnvData(string newData): 更新环境监测数据。
- string getEnvReport(): 生成环境数据报告。
- void warningAlert(): 触发环境异常警报。

11. **Fish** 类: 传感器监测到的鱼群类。由于传感器数据可能来自不同日期, 因此在系统中具体日期和鱼群类相对于一个键值对。

(a) 属性:

- string fishBreed: 鱼的品种名称。
- char growthStage: 鱼的生长阶段。
- int density: 鱼的密度值。
- double avgLength: 鱼的平均长度。

- double avgWeigth: 鱼的平均重量。

(b) 方法:

- void updateFishData(string newData): 更新鱼群的监测数据。
- string getFishReport(): 生成鱼群数据报告。

4.3.2 类图中类间关系解释

类图中类/接口之间的关系主要考虑以下几点:

1. **继承关系**: 由于所有用户都具有查看数据的功能, 因此设置了 Visitor 访客类作为所有用户的父类。在访客类之下, 由于还需要记录用户的注册信息, 因此设置 RegisteredUser 作为 Visitor 的子类, 继承了查看系统中数据的功能, 同时又新增了注册信息等一系列功能。在注册用户之下, 我们派生出了几个具体的用户类型 (数据分析师类、养殖户类、管理员类和政府职员类)。每个类都继承自注册用户类, 这表明它们共享一些基本属性和方法。并且可能有各自特定的方法和功能来满足不同的角色需求来贴合实际情况, 更好地维护系统。
2. **接口实现关系**: 为了统一管理用户登录方式, 系统将用户名密码登录和手机号登录功能抽象为 ILogin 接口。该接口要求所有继承的用户类都必须实现这些登录功能, 确保系统登录机制的一致性和规范性。
3. **聚合关系**: Environment 环境类作为聚合类, 整合了气象和鱼群数据, 构成完整的环境监测数据体系。虽然气象和鱼群数据作为环境数据的组成部分, 但它们各自保持独立性, 能够单独存在和使用。
4. **关联关系**:
 - 养殖户-传感器关系: 一对多关联, 养殖户可控制多个设备 (数量可为 0)。
 - 用户-数据关系: 多对多关联, 用户可查询多组环境/气象/鱼群数据 (数据可为 0)。
 - 数据-传感器关系: 多对一关联, 单台设备可采集多组数据 (设备可为 0)。

5 顺序图

5.1 顺序图简介

UML 顺序图

UML (统一建模语言) 顺序图 (Sequence Diagram) 是一种用于描述对象之间交互的行为图, 强调消息在时间维度上的顺序。它在软件工程中被广泛应用于需求分析、系统设计和测试用例编写等阶段。顺序图通过可视化的方式展现系统中对象之间的动态交互, 有助于开发人员、测试人员和其他相关人员更好地理解系统的行为。其主要的优势在于按时间的顺序表示对象之间的消息, 非常直观。

顺序图的基本组成元素

- **参与者 (Actor) 与对象 (Object)**: 参与者通常代表系统外部的用户或其他系统, 而对象则是系统内部的实体。通常以矩形框表示, 并沿水平方向排列。

- **生命线 (Lifeline)**: 从参与者或对象的矩形框下方延伸出的垂直虚线，表示该实体在交互过程中的存在周期。
- **激活 (Activation)**: 表示对象正在执行某个操作的时间段，通常以叠加在生命线上的细长矩形表示。
- **消息 (Message)**: 对象之间的通信，通常以箭头表示，箭头上标注消息的名称。
- **注释 (Note)**: 用于对图中的元素进行解释或补充说明，通常以带有折角的矩形表示，并通过虚线连接到相关元素。

5.2 顺序图构建

5.2.1 用户登录

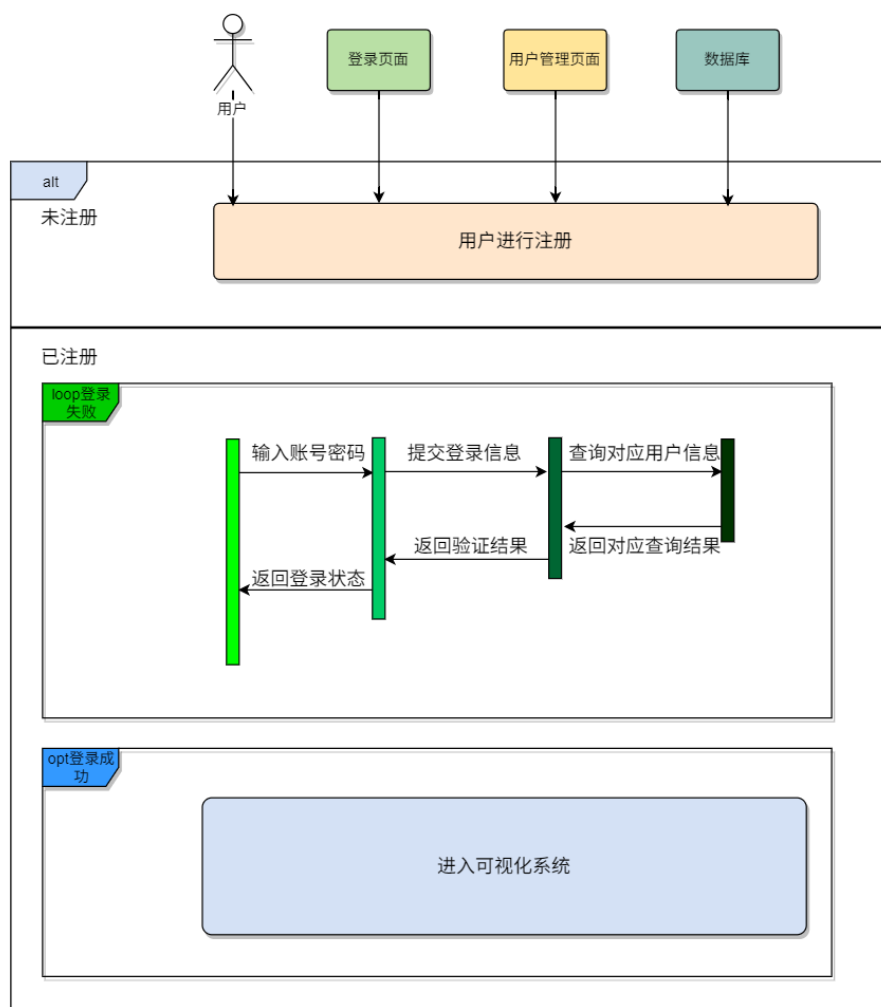


图 5.14: 用户登录图

如图 5.14 所示，描述了用户注册和登录的过程。

5.2.2 数据采集

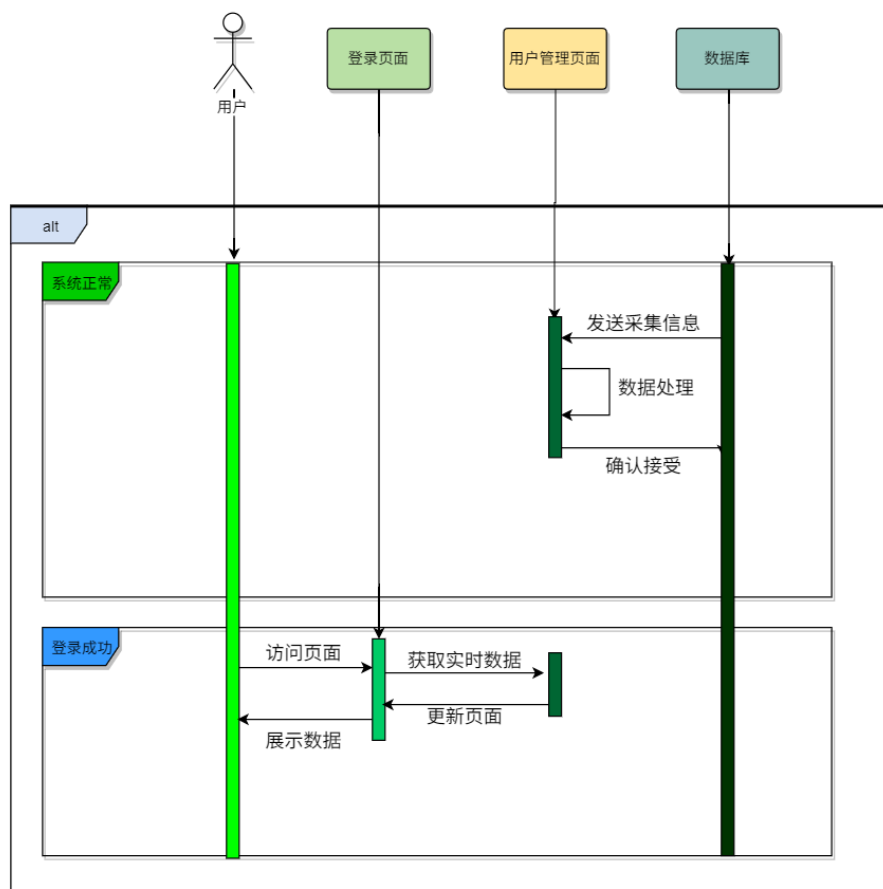


图 5.15: 数据采集

如图 5.15 所示，描述了系统数据采集和展示的过程。

5.2.3 报警设置

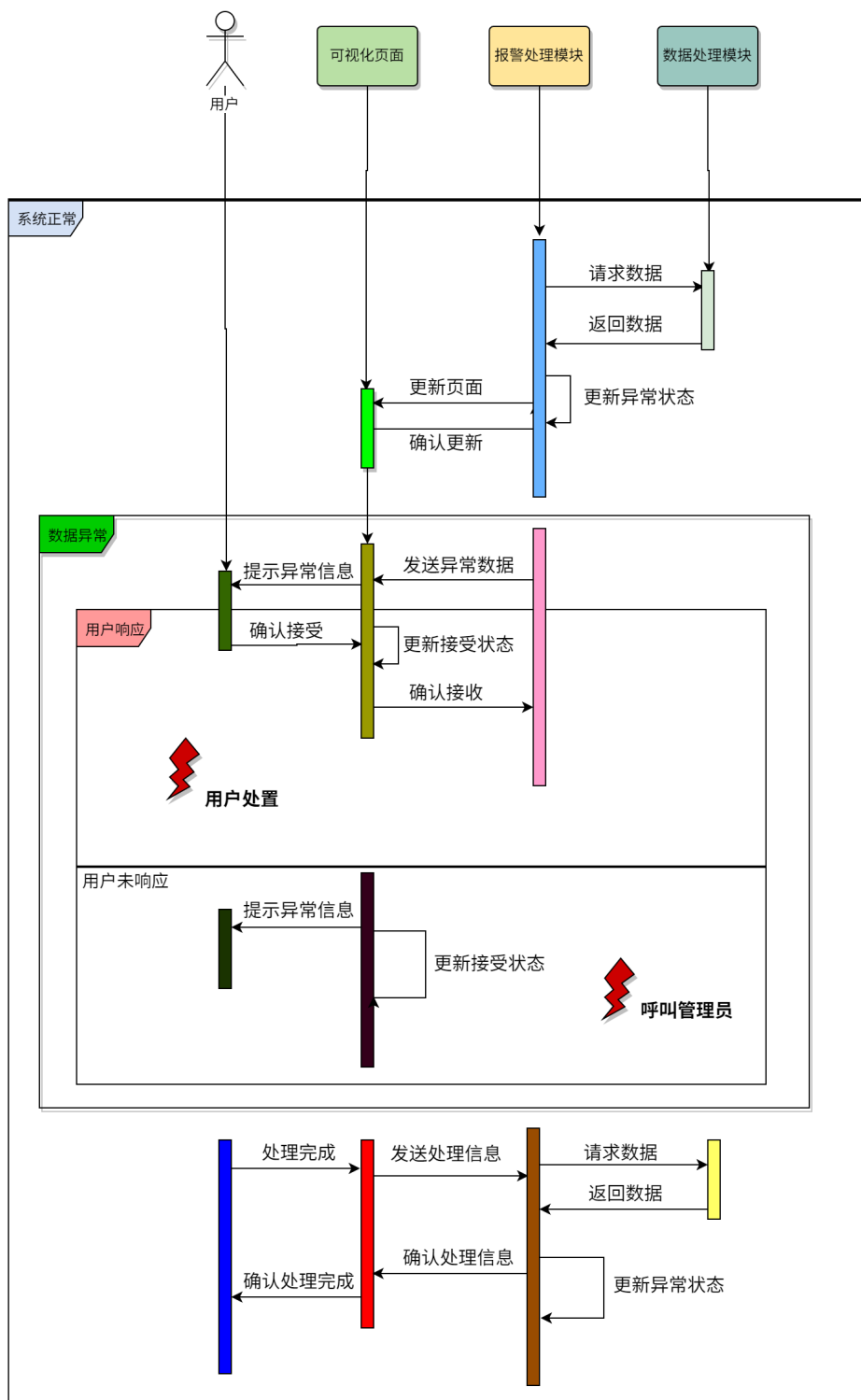


图 5.16: 报警设置

如图 5.16 所示，描述了系统报警和处理的过程。

5.2.4 系统管理

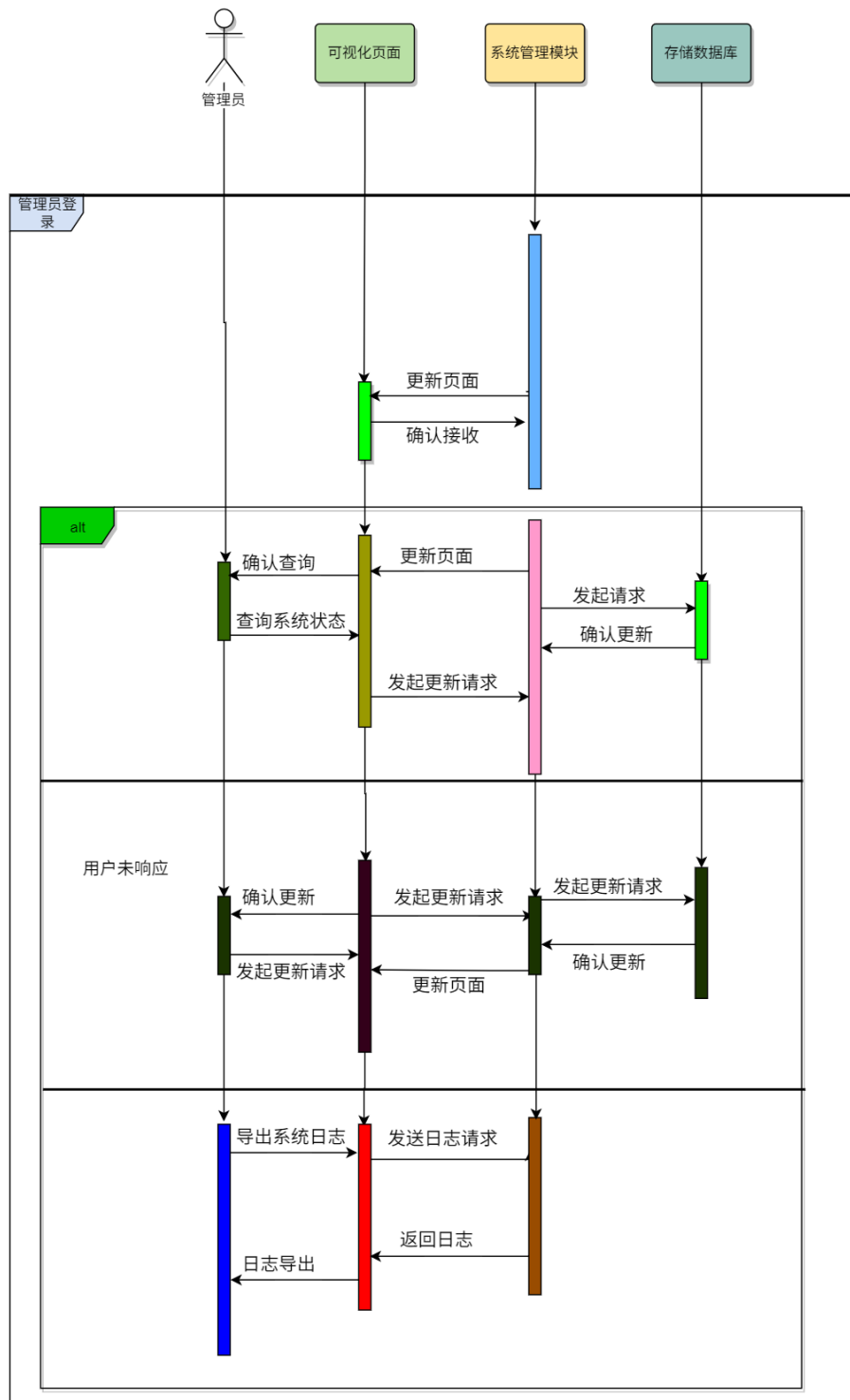


图 5.17: 系统管理

如图 5.17 所示，描述了系统管理的过程。

5.3 顺序图解释

5.3.1 用户登录

用户登录解释

新用户会被引导完成注册流程，已注册用户则直接进入登录验证环节。用户需在登录界面填写账号、密码等凭证信息，这些认证数据会被提交给用户管理模块处理。该模块向数据库发送查询请求，数据库引擎接收到请求后，会对账户信息进行检索，并将检索结果反馈给用户管理模块。

用户管理模块会把验证结果返回给登录界面进行展示。若验证未通过，系统会一直处于登录循环状态，直至认证成功为止。通过验证后，用户即可获得系统访问权限，进而进行数据查询、告警设置等后续操作。

5.3.2 数据采集

数据采集解释

在系统正常运行状态下，传感器会将采集到的数据传输至数据处理模块。该模块负责接收数据并执行处理操作，剔除其中的错误信息后，完成对数据库内容的更新。

当用户完成登录验证后，可视化页面会向数据处理模块发起最新数据请求。在获取返回的信息后，可视化页面将进行内容更新，并通过可视化方式向用户展示关键数据内容。

5.3.3 报警设置

报警设置解释

系统运行时，报警处理模块会持续监控数据处理模块的最新数据状态。一旦检测到异常，系统会立即将异常信息推送至可视化界面，提醒用户进行处理。若用户及时确认处理，系统不会触发管理员介入；若用户超时未响应，系统将自动通知管理员开展应急处理。

异常处理完成后，用户提交处理完毕请求，报警处理模块会重新校验数据状态并更新系统。确认异常解除后，系统恢复正常运行，继续循环监测数据并展示报警信息，确保异常情况能够得到及时响应和闭环处理。

5.3.4 系统管理

系统管理解释

管理员登录系统后，可视化页面会实时呈现最新数据及系统参数。系统管理模块与数据处理模块保持持续交互，动态获取并更新这些信息。

此顺序图展示了三种典型的管理员维护操作：查询系统内容、更新系统数据和导出系统日志。管理员通过可视化界面发起请求后，系统管理模块会对操作进行处理——或向数据处理模块查询信息，或直接导出存储的日志数据。整个交互流程保障了系统维护的高效性与数据的准确性。

6 协作图

6.1 协作图简介

UML 协作图

UML 协作图（也称为通信图）是 UML 中用于表示对象之间交互的一种结构图。它与顺序图一样，都属于交互图，但协作图强调的是对象之间的结构关系以及消息传递的顺序。协作图通过对象和它们之间的连接表示交互结构，适合用于可视化对象之间的通信结构。

协作图主要用于分析系统的结构性动态行为，常用于建模系统的用例行为、逻辑流程或模块间协作。

协作图的基本组成元素：

- **对象 (Object)**：以矩形框表示，框中标注对象名称，一般格式为对象名: 类名。若省略类名，默认仅表示某个对象实例。
- **链路 (Link)**：表示对象间的关联关系，是对象之间能够进行通信的前提，通常为直线连接两个对象。
- **消息 (Message)**：表示对象之间的调用行为，标注在链路上，通常带有序号，以箭头方向表示消息发送方与接收方。消息格式为序号: 消息名称 (参数)。
- **顺序号 (Sequence Number)**：用来表示消息发生的先后顺序。可以使用嵌套的分段编号（如 1, 1.1, 1.2）表示消息的层级调用关系。

6.2 协作图构建

协作图与顺序图类似，主要反应对象之间的结构关系以及消息传递的顺序，对于顺序图部分的功能要点对应展示如下：

6.2.1 用户登录



图 6.18: 用户登录

如图 6.18 所示，展示了用户登录的过程中的对象结构关系。

6.2.2 数据采集

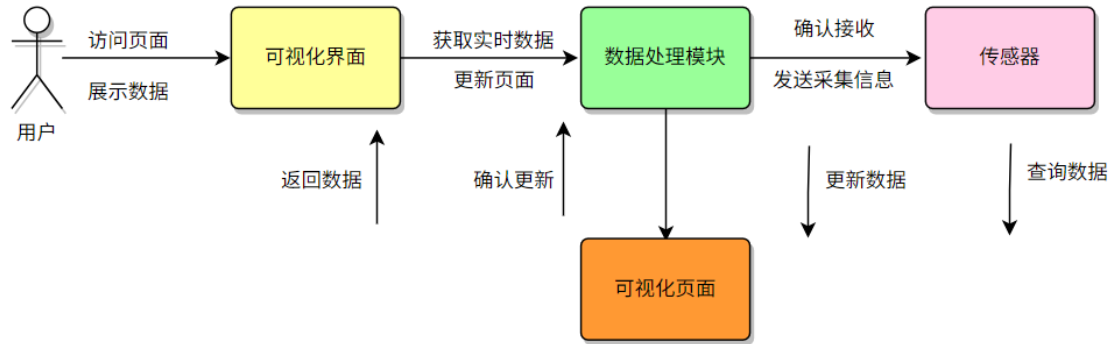


图 6.19: 数据采集

如图 6.19 所示，描述了系统数据采集和展示的过程中的对象结构关系。

6.2.3 报警设置

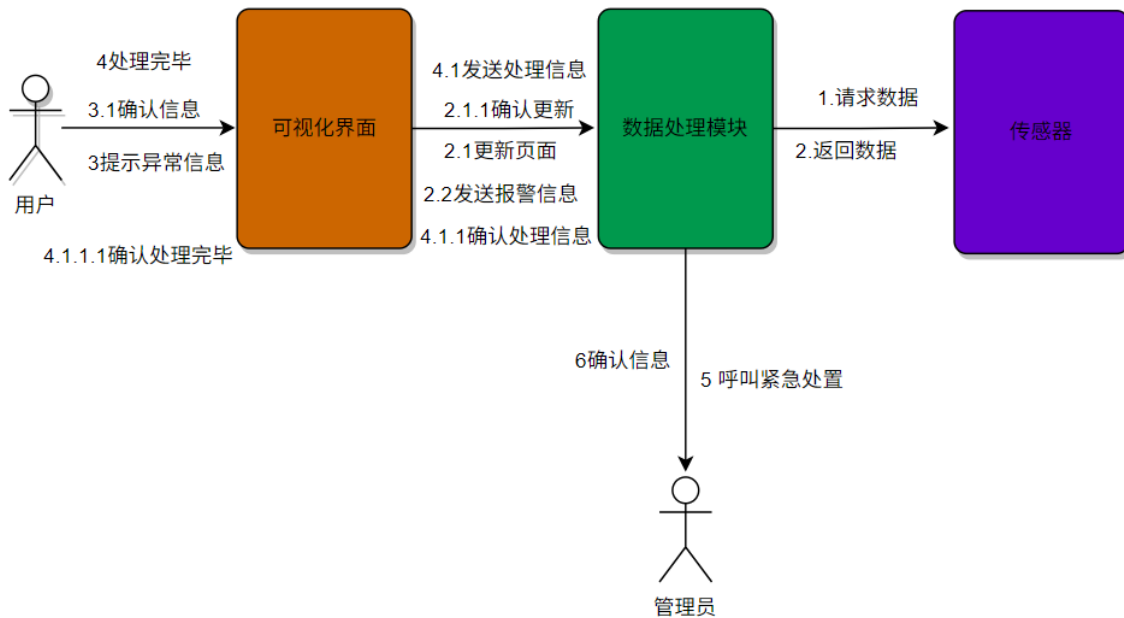


图 6.20: 报警设置

如图 6.20 所示，描述了系统报警和处理的過程中的对象结构关系。

6.2.4 系统管理

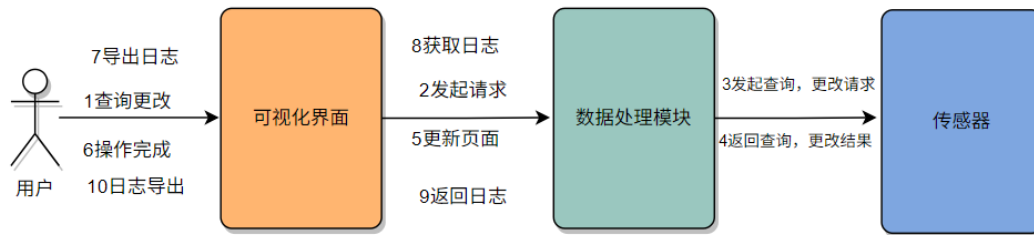


图 6.21: 系统管理

如图 6.21 所示，描述了系统管理的过程中的对象结构关系。

6.3 协作图解释

6.3.1 用户登录

用户登录解释

用户通过登录页面提交账号信息后，系统启动分层验证流程。登录页面作为安全网关，将用户请求转发至用户管理模块，并以清晰的状态信息向用户反馈验证结果。验证通过后，系统自动跳转至可视化主界面，实现访问流程的无缝过渡。

在后台处理中，用户管理模块与数据库紧密协作完成核心凭证校验。该模块接收登录请求后执行数据比对，生成包含验证结果的响应报文。整个验证过程采用模块化设计，既保障了前端交互的简洁性，又强化了后端处理的安全性，形成高效的登录验证闭环。

6.3.2 数据采集

数据采集解释

数据采集环节由传感器与数据处理模块协同完成。传感器负责采集原始数据，数据处理模块对数据进行清洗、转换等处理，并与数据库交互实现数据的存储更新，形成完整的数据采集闭环。数据展示环节通过可视化页面与数据处理模块联动实现。当用户发起请求时，可视化页面从数据处理模块获取数据，数据处理模块再向数据库查询所需内容，最终完成数据的可视化呈现，确保用户能够直观获取信息。

6.3.3 报警设置

报警设置解释

报警处置系统采用分层消息处理架构，需确保报警信息实时推送并获取用户反馈，形成高效的信息传递闭环。

当报警触发时，系统优先等待用户响应处理。若用户未及时响应，系统将自动升级至管理员应急处理流程。这种分级响应机制既保障了常规报警的处置效率，又通过管理员的兜底介入确保了系统稳定性和生产安全。

6.3.4 系统管理

系统管理解释

管理员作为系统安全的核心责任人，被赋予更高权限以执行关键操作，例如系统信息查询、信息更新及运行日志导出。这些操作通过可视化界面作为交互枢纽，由系统管理模块统一调度处理，形成标准化的管理流程。

在具体实现中，系统采用分层调用机制：常规管理指令由系统管理模块直接处理，涉及数据操作时则调用数据处理模块的专用接口。这种设计既保持了与普通用户操作流程的一致性，又通过模块化接口实现了权限隔离，确保管理操作的安全性和可靠性。

7 状态图

7.1 状态图简介

UML 状态图（亦称状态机图）用于刻画对象在整个生命周期中所经历的状态变化、状态间的迁移路径，以及对象如何响应特定事件。此类图形特别适用于描述状态切换频繁或依赖外部事件驱动的系统逻辑，有助于开发者更好地掌握对象在各种场景下的行为反应。

状态图的核心构成要素包括：

- **状态 (State)**：表示对象在某一时刻所处的情形，通常绘制为圆角矩形。
- **初态 (Initial State)**：表示对象生命周期的起始点，图中以实心圆描绘。
- **终态 (Final State)**：代表对象生命周期的结束，常用一个内部填充黑色的同心圆表示。
- **转换 (Transition)**：用于表示状态之间的切换，以带箭头的线段连接，箭头指向目标状态，同时可标注触发事件和相应动作。
- **事件 (Event)**：指引起状态迁移的外部条件或信号。
- **动作 (Action)**：在状态变更时触发执行的操作。

借助状态图，开发人员可以直观地观察对象如何根据不同事件做出响应，进而理清其在各个状态下可执行的行为逻辑。

7.2 状态图构建

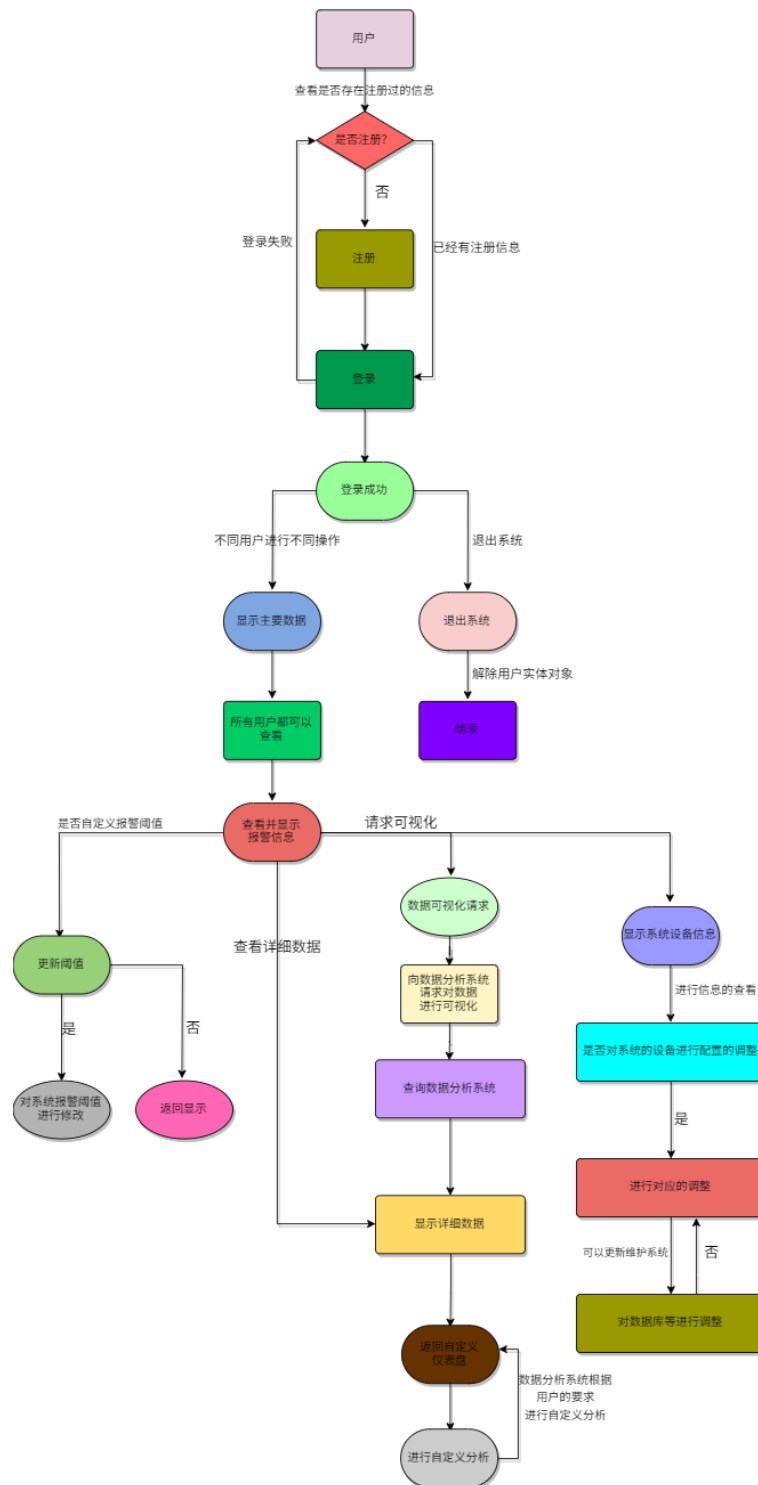


图 7.22: 状态图

7.3 状态图解释

本系统的状态图如图7.22所示，围绕系统中“用户”这一核心对象，系统状态将根据用户扮演的角色及其执行的操作发生转变。具体状态流程及其转换关系如下：

1. **初始状态**：系统启动后进入初始状态，处于等待用户输入阶段。
2. **用户验证阶段**：
 - **登录操作**：已注册用户输入凭证尝试登录。
 - **注册操作**：新用户需先进行账号注册。
 - 若登录失败，系统提示用户确认注册状态，并根据结果返回相应操作选项。
3. **角色驱动的操作阶段**：用户成功登录后，根据身份类别进入不同功能操作流程，主要包括：
 - **浏览核心数据**：所有用户均可访问的主要信息概览，如海洋牧场基本运行状况。
 - **获取详细数据**：养殖人员与数据分析者可查看深层数据，如环境因素、气象指标、鱼群状态等。若用户请求可视化展示，系统调用数据分析模块：
 - 若存在预设仪表盘，直接返回该视图；
 - 若无现成模板，依据用户请求生成新分析图表后展示。
 - **查看预警信息**：仅限养殖户与系统管理员查看的报警内容，包括异常情况、系统提示等。用户可选择是否设定自定义阈值：
 - 若选择设定，系统更新报警参数并重新展示；
 - 若不设定，直接显示现有报警信息。
 - **系统设备状态查看与配置**：包括对传感器、摄像头等硬件设备及系统整体状态的监控。若用户选择配置操作：
 - 管理员可进行系统级维护与参数设置；
 - 养殖户可远程控制设备，如摄像头启停、水温调节等；
 - 配置完成后，系统同步更新数据库并显示设备详情。
4. **系统退出**：用户完成操作后选择退出，系统释放相关用户实体与权限绑定，回到终止状态，标志交互流程结束。

8 构件图

8.1 构件图简介

UML 构件图用于展现软件系统中物理构成单元的静态结构视图，聚焦于系统构件的组成、依赖关系以及它们如何协作构建整体系统。该图广泛应用于模块化系统的设计中，有助于开发者把握系统的构架布局以及外部库或第三方模块的集成方式。

构件图主要由以下关键元素构成：

- **构件 (Component)**：表示代码的物理实现单元，如程序模块或库文件，具有独立部署与替换的能力。

- **接口 (Interface)**: 指构件向外界提供的服务说明, 图中以小圆连接符号表示, 并通过连线附着于提供接口的构件。
- **依赖 (Dependency)**: 体现一个构件对另一个构件服务或功能的依赖, 常以带箭头的虚线表示。
- **关联 (Association)**: 反映构件之间的信息传递或通信链路。
- **制品 (Artifact)**: 构件所关联的实际物理文件, 例如二进制可执行文件、动态链接库或自动化脚本。
- **节点 (Node)**: 指代构件的运行承载环境, 如服务器、虚拟机或数据库实例。

通过构件图, 开发团队能够理清系统构建块之间的组织与协同方式, 从而优化系统设计结构。同时, 该图也为系统部署与运维人员提供了部署方案与依赖配置的清晰视图。

8.2 构件图构建

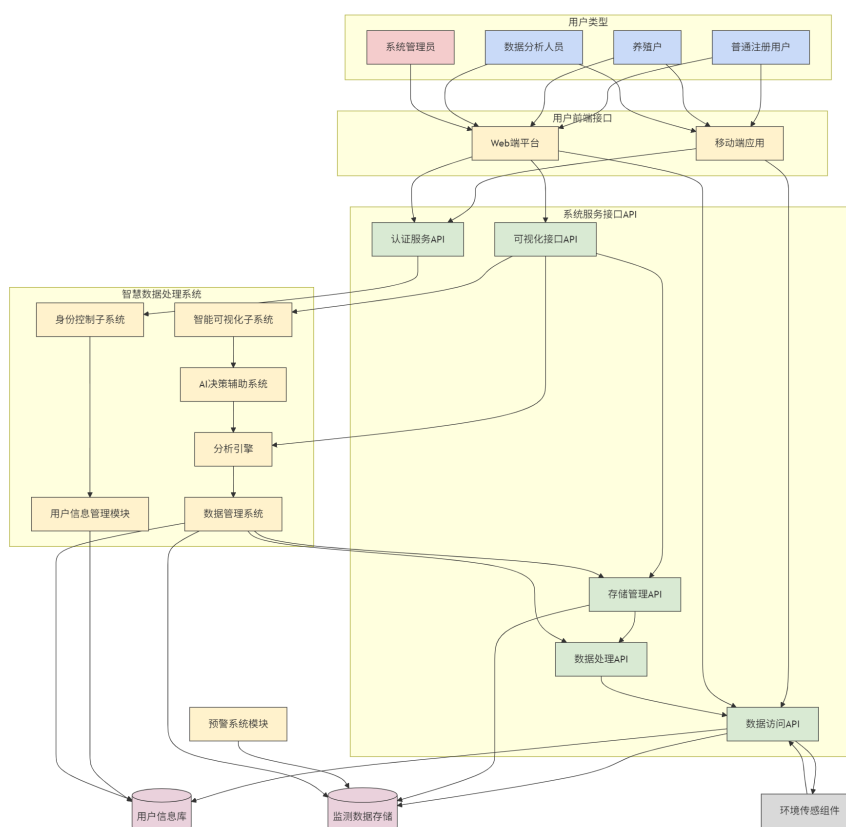


图 8.23: 构件图

8.3 构件图解释

海洋牧场智慧可视化监测系统的构件图如图8.23所示。为便于理解整体结构与模块间的依赖关系, 以下对图中各关键构件进行详细说明:

1. 用户类型

- **通用用户群体**：包含普通注册用户、养殖户和数据分析人员。普通用户可访问基础数据视图并接收通知推送；养殖户具备获取精细化监测报告、配置环境参数等权限；数据分析人员可对系统数据进行挖掘处理，为策略决策提供支撑。
- **系统管理员**：负责系统维护，包括用户账号管理、系统参数配置、数据更新与备份，同时监控系统状态与数据安全。

2. 环境传感组件

海洋牧场部署的传感设备用于采集环境指标、气候信息与鱼群活动数据。这些设备通过接口将数据实时传输至中央处理单元，作为系统数据分析与响应控制的基础。

3. 用户前端接口

- **移动端应用**：支持智能终端远程访问，使用户可随时查看牧场状态、接收系统预警并进行基础交互。
- **Web 端平台**：提供全面的交互界面，支持图形分析、报表生成与历史数据追踪等复杂功能。

4. 系统服务接口 API

- **认证服务 API**：处理身份验证、权限校验等功能，确保用户身份合法并控制数据访问权限。
- **数据访问 API**：为上层系统提供用户数据与传感器信息的读取功能，支持实时或存储数据访问。
- **数据处理 API**：承担数据清洗、格式标准化与初步统计任务，依赖数据访问 API 提供的原始数据。
- **存储管理 API**：将处理后的分析结果安全保存，为可视化、报警机制及 AI 模型调用提供数据支持，依赖数据处理接口。
- **可视化接口 API**：为用户呈现动态图表、实时仪表板等直观内容，涵盖阈值管理、交互图形等功能。

5. 智慧数据处理系统

- **身份控制子系统**：限制敏感功能的访问权限，仅允许合法授权用户操作。
- **用户信息管理模块**：维护用户账户、权限配置及个人资料。
- **数据管理系统**：实现数据的接收、初步加工及持久化存储。
- **分析引擎**：开展高阶数据分析与模式识别，提供决策参考，依赖前端数据处理支持。
- **AI 决策辅助系统**：运用智能算法进行趋势预测与策略推荐，依赖分析引擎结果。
- **智能可视化子系统**：按需生成个性化可视化界面以满足不同用户需求。

6. 预警系统模块

该模块负责阈值监控与异常检测。一旦触发预警条件，系统立即推送通知至相关人员，实现快速响应与问题定位，依赖监测数据库的数据支持。

7. 数据库系统

- **监测数据存储**：集中管理从各类传感器或外部数据源收集的原始与处理后信息。
- **用户信息库**：保存用户注册资料、权限设定及个性化配置参数。

9 部署图

9.1 部署图简介

UML 部署图主要用于描述软件系统在物理层面上的部署结构。它展示了系统运行所依赖的硬件资源（如服务器、终端设备等）以及这些硬件节点上部署的软件构件的具体安排。该图在展示系统的网络拓扑与部署架构方面具有重要价值，能够辅助理解软硬件之间的映射关系。

部署图中主要包含以下元素：

- **节点 (Node)**：表示物理或虚拟的运行平台，如服务器、主机或云实例，图中通常以立方体形状表示。
- **构件 (Component)**：指部署在节点上的软件模块或服务实例，体现软件在物理设备上的分布情况。
- **制品 (Artifact)**：部署在节点中的实际文件或物理单元，如可执行程序、数据库文件或配置脚本。
- **关联 (Association)**：用于表达节点之间的通信路径或数据交互链路，代表系统中的网络连接。
- **依赖 (Dependency)**：表示一个节点或制品在运行时对其他构件、节点或制品的依赖关系。

借助部署图，设计人员与系统运维团队可清晰把握整个系统在物理环境中的运行结构，明确软件部署方式与各组件之间的通信机制，为系统实施、部署、扩容及维护提供了重要的指导依据。

9.2 部署图构建

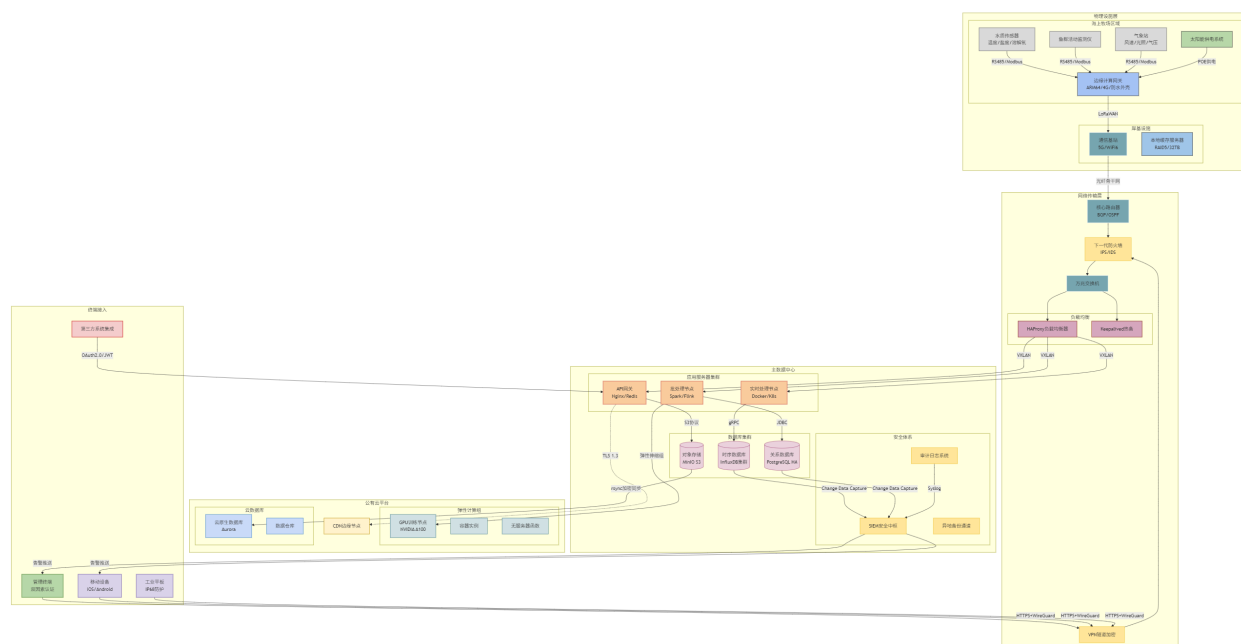


图 9.24: 海洋牧场智慧可视化监测系统的部署图

9.3 部署图解释

海洋牧场智慧可视化监测系统的部署结构如图9.24所示。该图展示了系统在物理设备层面的分布情况，涵盖从数据中心到现场设备、移动接入以及云平台的整体网络架构。以下将对部署图中各部分组件逐一进行说明：

1. 数据中心

- **中央服务器：**作为系统处理核心，承担所有监测数据的接收、处理与分析任务，支持实时监控、预测计算与报告生成。服务器需具备高性能以应对高并发请求及大规模数据流处理。
推荐配置：8 核 CPU、64GB 内存、2TB SSD。
- **数据库系统：**负责存储所有关键数据，包括原始传感数据、分析结果及用户资料等。要求具备高写入性能、良好的数据一致性与安全保障能力。
建议数据库：PostgreSQL 或 InfluxDB。

2. 办公室环境

- **管理终端：**为系统管理员提供操作界面，用于查看系统运行状态、设置系统参数、处理告警事件与权限管理等。界面应具备良好的信息展示能力，推荐使用高分辨率或多屏配置。
最低配置：1080p 显示器。

3. 现场设施

- **环境传感器**：部署在海上牧场，用于采集水体环境指标（如温度、盐度、溶解氧等），要求设备耐腐蚀、精度高、适应恶劣海洋环境。
- **数据采集模块**：位于现场的数据中转节点，连接传感器与服务器，负责数据打包与发送。其设计应兼顾低功耗与高稳定性，支持无线或有线传输方式。

4. 移动访问终端

- **移动设备**：包括智能手机和平板电脑，用于远程访问系统数据与控制界面，支持实时查看与告警接收，具备跨平台访问能力。
支持系统：iOS 与 Android。

5. 网络接口

- **数据传输通道**：连接数据中心与外部网络，确保信息高速流通。应保障海量数据的上传与用户请求的响应能力。
推荐带宽：最低 1Gbps，建议 10Gbps 以太网。

6. 云计算平台

- **云服务节点**：在系统负载剧增时提供弹性计算资源与额外存储空间，支持数据备份、异地灾备与模型训练等扩展任务。
推荐系统环境：Ubuntu Server (LTS 版本)。

系统整体通信通过 HTTPS 等加密协议进行客户端-服务器交互，确保传输链路的数据安全；服务器与数据库之间采用高效连接方式（如 JDBC）以保障低延迟访问。安全机制方面：

- 使用多因素身份验证（用户名、密码 + 移动设备验证）以提升账户安全；
- 数据传输过程采用 SSL/TLS 协议加密，防止信息泄露；
- 系统引入访问控制策略，限定用户权限范围；
- 所有组件均经过安全配置与稳定性测试，支持系统长期稳定运行与弹性扩展。

综上所述，系统各物理与逻辑组件紧密配合，构建出一个高性能、高安全、易扩展的智能化监控平台，全面满足海洋牧场的数字化管理与远程控制需求。